THREAT_MODEL

Status: Draft v1. Scope: public web app, open signup, repo code is read but never executed.

1. Assets

User GitHub tokens. AI provider keys. Users' repo snapshots and V2. The integrity of users' GitHub repos. Server availability and AI budget.

2. Attackers

A malicious user (crafted repo, abuse, trying to read others' data). A user who tries to push to someone else's repo. A stolen session. Hostile text inside a repo (prompt injection). Automated abuse.

3. Core rule

No repository code is ever executed: no install, build, test, lint, scripts or git hooks. If this rule breaks, the old sandbox problem returns. A test must prove it.

4. Threats and controls

Format: threat, control, how it is enforced, how it is tested.

ID	Threat	Control	Enforced by	Test
T1	OAuth login hijack or CSRF	State parameter, HTTPS, secure httpOnly sameSite cookie, short session life	Auth module, middleware	Callback with wrong state is rejected
T2	GitHub token theft	Encrypt at rest, never in responses or logs, backend only	Token vault module, log scrubber	Search responses, logs and DB dumps for token patterns

ID	Threat	Control	Enforced by	Test
T3	User selects other person's repo, org repo or private repo	Server checks owner id equals user id, ownerType User, private false, using GitHub API	Repo module	Direct API calls with forged repo names are rejected
T4	Git hooks or config run code during clone	Clone with hooks disabled, no submodules, safe config, never run repo commands	Clone helper	Fixture repo with hooks and scripts must produce no side effect
T5	Symlink or path traversal to read server files	Do not follow symlinks, resolve and verify every path stays inside the snapshot folder	File reader and editor	Fixture with symlink to /etc and ../ paths
T6	Zip bomb or huge repo exhausts disk or memory	Limits on total size, file count, file size, clone time	Clone helper	Oversized fixture returns SNAPSHOT_TOO_LARGE
T7	Malicious file crashes parser (static checks)	Parse with timeouts and size caps, in the worker only, errors contained	Static check module	Pathological file fixtures

ID	Threat	Control	Enforced by	Test
T8	Prompt injection from repo text	Treat repo as data in prompts, no tools or secrets for AI, validate AI output with schema, scope check against snapshot files, AI cannot approve or change state	AI client, plan validator	Fixture repo saying ignore rules and edit other file must not widen scope
T9	AI suggests edits to unplanned or sensitive files	Operations restricted to snapshot files and to user-selected operations. Extra confirmation for high-risk and delete	Plan validator, approval check	Out-of-scope operation rejected
T10	Stale or replayed approval	Approval bound to user, workflow, SHA, op ids and plan hash. Single use.	Approval manager	Change SHA then approve or execute gets rejected
T11	Double click or race causes double execute or push	Conditional state updates, unique keys, deterministic job ids	Workflow module	Concurrent request tests

ID	Threat	Control	Enforced by	Test
T12	Push to main, force-push, auto-merge	These code paths do not exist. Target branch name is generated by server and checked not equal to main. PR base is main but merge is never called.	Delivery module	Static search plus test that main target is refused
T13	Push to wrong repo or after repo changed	Re-verify repo, owner, public status and remote SHA before every commit and before opening the PR	Delivery module	SHA changed fixture gives STALE
T14	Over-broad GitHub token scope	Write access is needed from login because work is saved on a GitHub branch. Prefer the narrowest option (a GitHub App limited to chosen repos) if GitHub docs confirm it. A classic OAuth write scope may cover all the user's public repos. Exact scopes to be verified in GitHub docs.	Auth module	Scope check on callback

ID	Threat	Control	Enforced by	Test
T15	One user reads another user's data	userId in every query, ownership check in every route, unguessable ids	Data access layer	Cross-user access returns 404 or 403
T16	Abuse: many workflows, bots	Rate limits per IP and user, daily workflow cap, concurrent cap, ban	Middleware	Load tests
T17	AI cost exhaustion	Per-user AI usage limit, input size caps, global budget alarm	AI client, usage counters	Limit reached returns AI_LIMIT
T18	Secrets leak into logs or audit	Log scrubber, no request bodies with code in logs	Logger	Log scan test
T19	Service to service abuse (aiServices called directly)	aiServices not public, shared secret or private network, input validation	aiServices config	Direct call without secret rejected
T20	Session theft via XSS	React escaping, no raw HTML rendering of repo content, Content Security Policy, httpOnly cookie	Frontend and headers	Repo file with script tags renders as text
T21	Malicious content shown in diff or findings (HTML or markdown injection)	Render as plain text or sanitized	Frontend	Same as above

ID	Threat	Control	Enforced by	Test
T22	Dependency compromise in Obsidian itself	Pin versions, lockfiles, npm audit and pip audit, review new dependencies	Dev process	Audit report
T23	Incomplete deletion of user data	Delete DB records and disk files, verify, handle active jobs first	Retention module	Check folder and records after delete
T24	Misleading users about safety	Review page states code was not run and not tested. Never say secure or production-ready.	Frontend copy	UI test for warning text

5. Residual risks (honest list)

- Static analysis cannot prove code works or is secure.
- AI can still suggest harmful or wrong edits. The human approval is the final control, so the review page must be clear.
- A malicious repo may contain code that is harmful when the user later runs it. Obsidian only reports what it can see.
- Parser bugs are possible, so parsing is isolated to the worker with limits.
- Hosting and OAuth details are not yet verified. A stolen token now has write access to the user's public repos, so token theft has higher impact. Work branches on a public repo are visible to everyone, including edits not yet reviewed.

6. Security acceptance criteria

All tests for T3 to T13 and T15 pass. A fixture repo with malicious hooks, scripts and symlinks causes no execution and no file read outside the snapshot. No secrets found in logs. Independent review is recommended before public launch.